Spring Boot 個人演習ガイド

書籍管理システム

目次

- はじめに p.5
- 準備：演習環境のセットアップ p.6
  - a. プロジェクトのインポート（Eclipse） p.6
  - b. データベースのセットアップ p.6
- 演習2： Spring Web アプリケーション（画面遷移とデータ通信） p.7
  - 課題2.1：「ログイン（トップ画面）」の作成と画面遷移 p.7
  - 課題2.2：「書籍管理メニュー画面」の作成 (リンクと検索モック) p.8
  - 課題2.3：「書籍登録」のモック作成 (POST通信とFormクラス) p.10
- 演習3：Spring Data JPA（データベースとの連携） p.11
  - データベースのテーブル定義 p.11
  - 課題3.1：エンティティクラスの作成 p.13
  - 課題3.2：リポジトリインターフェースの作成 p.14
  - 課題3.3：Service層の作成 p.15
  - 課題3.4：全件一覧と検索モックのDB連動化 p.16
  - 課題3.5：書籍詳細画面の実装（更新・削除の起点） p.17
  - 課題3.6：新規登録機能の実装（モックからDB保存へ） p.18
  - 課題3.7：更新機能と削除機能の実装 p.19
- 第4章：入力チェックとログイン認証（セッション管理） p.20
  - 課題4.1：入力チェック（Validation）の実装 p.20
  - 課題4.2：ユーザー管理用のEntityとRepository作成 p.22
  - 課題4.3：ログイン・ログアウト機能の追加 p.23
- 演習5：共通レイアウトの適用とデザインの統一（Thymeleaf Layout Dialect） p.24
  - 課題5.1：ライブラリの導入 p.24
  - 課題5.2：全画面へのレイアウト適用（リファクタリング） p.25
  - 課題5.3：ログイン・ログアウトボタンの切り替え（th:ifの活用） p.26
- 演習6：高度なデータ連携とユーザー体験の向上（動的な項目取得と0件制御） p.27
  - 課題6.1：検索結果0件時の表示制御 p.27
  - 課題6.2：登録画面での動的なジャンル選択（GenreServiceの作成） p.28
  - 課題6.3：【発展】JPQLを用いた複合検索の実装 p.29
  - 課題6.4：【オプション】更新画面への動的リスト適用 p.29
- 第7章： デザインの洗練（Bootstrapの導入） p.32
  - 第7章でよく使う Bootstrap クラス早見表 p.32
  - 課題7.1： Bootstrap の導入と既存CSSの排除 p.32
  - 課題7.2： ボタンのデザイン統一 p.32

- 課題7.3： フォーム部品の整形 p.32
- 課題7.4： テーブル（書籍一覧）の整形 p.33
- 課題7.5： 全体のレイアウト刷新 p.34

Spring Boot 個人演習ガイド

書籍管理システム

はじめに

本演習は、Spring Boot入門の研修終了翌日に、1日かけて取り組む総合演習です。
テキストで学習した順序に沿って**「書籍管理システム」**の機能を少しずつ作成・拡張していきます。最終的には、CRUD操作からログイン機能までを備えたWebアプリケーションが完成します。

> **Note**
>
> **【章立てについて】**
> 本ガイドは「演習2」から始まります。これは、Spring Boot入門の講義テキストにおいて、本格的な実装解説が第2章から開始されることに合わせているためです。演習番号とテキストの章番号を一致させることで、各課題に取り組む際にテキストのどの部分を参照すればよいかを分かりやすくしています。

> **Tip**
>
> **【学習のヒント：全体像の確認】**
> 実装を始める前に、各章のステップでシステムがどのように進化していくかを 書籍管理システム補足資料（画面遷移・イメージ図・ER図） で確認できます。スクリーンショットを通して、最終的なゴールのイメージを掴みましょう。

準備：演習環境のセットアップ

演習を始める前に、配布された演習資料（zipファイル）を解凍し、以下の手順で開発環境を整えてください。

1. プロジェクトのインポート（Eclipse）

1. 配布された zip ファイルを任意の場所へ解凍します。
2. Eclipse を起動し、メニューの **[ファイル] > [インポート]** を選択します。
3. **[Maven] > [既存 Maven プロジェクト]** を選択して `[次へ]` をクリックします。
4. [ルート・ディレクトリー] の `[参照]` ボタンを押し、解凍したフォルダ内の `TrainoBook` フォルダを指定して `[完了]` をクリックします。

2. データベースのセットアップ

1. 解凍したフォルダ内の `dbsetup` フォルダを開きます。
2. `dbset.bat` をダブルクリックして実行します。
3. 黒い画面（コマンドプロンプト）が開き、演習用データベースの構築が開始されます。**正常に終了したことを確認して**画面を閉じます。

演習2： Spring Web アプリケーション（画面遷移とデータ通信）

> **【学習のねらい】**
> 第2章の知識（Controller、GET/POST、Formクラス、Model）を使って、書籍管理システムの「UI（画面）」と「Java側へのデータ送信」のモック（仮動作）を作成します。データベースへの保存・検索は次章で行うため、本章では**「入力したデータが画面遷移後に正しく表示されるか」**を確認します。

課題2.1：「ログイン（トップ画面）」の作成と画面遷移

プロジェクト実行時にシステム入り口となる「ログイン画面」を作り、特定の画面へ遷移する処理を作成しましょう。

1. `jp.co.trainocate.book.controller` パッケージを作成し、`LoginController` クラスを作成してください。これに `@Controller` アノテーションを付与します。
2. URL `http://localhost:8080/` にアクセスした際、`index.html` を返すように **GET マッピング** のメソッドを作成してください。
3. `src/main/resources/templates` の中に `index.html` （ログイン画面のHTMLファイル）を作成し、ページタイトルを「ログイン（書籍管理システム）」としてください。
4. `index.html` 内に、以下の構成でログイン用の `<form>` を作成してください。
   - **送信先（action）**：`/login`
   - **送信方法（method）**：`POST`
   - **入力項目**：ユーザID（`name` 属性を `userId` にする）、パスワード（`name` 属性を `password` にする）
   - **送信ボタン**：「ログイン」
5. `LoginController` に `/login` を受け取る **POST マッピング** のメソッドを作成し、引数で `userId` と `password` を受け取るようにしてください（第2章では認証処理のモックとして、受け取るだけで次に進みます）。
6. このメソッドの遷移先として、リダイレクトはさせずに、直接 `book_index.html` へ画面遷移するようにし、この後作成する「書籍管理メニュー画面」へそのまま移動させます。

課題2.2：「書籍管理メニュー画面」の作成 (リンクと検索モック)

ログイン成功後に表示される「書籍管理メニュー画面（`book_index.html`）」を作成し、他機能へのリンクや検索フォームを配置します。

1. `BookController` クラスを作成（`@Controller`付与）し、`/book/index` に対するGETマッピングを用意して、画面 `book_index.html` を返すようにします。
2. `src/main/resources/templates` の中に `book_index.html` （メニュー画面のHTMLファイル）を作成し、まず以下の2つのリンクを配置してください。
   - ①「全書籍リストの確認」への遷移リンク
   - ②「書籍情報の登録」への遷移リンク

   > **【ヒント】** Thymeleafを用いたリンクの作成方法は、5章を参照してください。書き方は以下のようになります。
   > `<a th:href="@{/book/list}">全書籍リストの確認</a>`

3. ①のリンク先として動作させるため、`BookController` に `/book/list` のGETマッピングを追加し、遷移先の `book_list.html` （書籍リスト画面のHTMLファイル）を作成してください。この画面はモックとして「書籍一覧画面（※本来はここに本の一覧が出ます。第3章で作成します）」といった文言のみ表示しておいてください。
4. 再び `book_index.html` の中に戻り、以下の2つの検索用の `<form>` を作成してください。すべて `GET` メソッドで送信します。
   - **① 書籍名検索フォーム**：
     - 入力項目：テキストボックス（`name`属性を `keyword` にする）
     - 送信先（action）：`/book/search/title`
     - 送信ボタン：「タイトルで検索」
   - **② 価格検索フォーム**：
     - 入力項目：最低価格の数値（`name`属性を `minPrice` にする）、最高価格の数値（`name`属性を `maxPrice` にする）
     - 送信先（action）：`/book/search/price`
     - 送信ボタン：「価格帯で検索」
5. `BookController` に上記の検索用送信先（`/book/search/title`、`/book/search/price`）に対応するメソッドを作成してください。
6. Controllerの引数でそれぞれ送信されたパラメータを受け取り、`Model` に格納してください。
7. 遷移先の画面として、`src/main/resources/templates` の中に `book_search_result.html` （検索結果表示のHTMLファイル）を作成してください。本章ではモックとして、Controllerから渡されたパラメータを表示するだけに留めます。

- **【出力例】** 「タイトルキーワード：$keyword$」や「価格：{minPrice}円〜${maxPrice}円」など、受け取った値が画面に出ればOKです。
  (※実際の検索結果一覧を表示したり、高度な表示制御を行ったりするのは、後の章（第3章、第6章）で行います)

課題2.3：「書籍登録」のモック作成 (POST通信とFormクラス)

画面のリンクから登録画面へ遷移し、Formクラスを使って一括でデータを受け取る練習をします。

1. `jp.co.trainocate.book.form` パッケージを作成し、 `BookForm` クラスを作成してください。
2. フィールドとして `title` (String)、 `author` (String)、 `price` (Integer) を用意し、Lombokの `@Data` を付けてください。
3. `BookController` に `/book/form` 用の GET マッピングを作成し、遷移先の `book_form.html` （書籍登録用HTMLファイル）を作成して入力画面を作ってください。
   - *(※この画面は、課題2.2で作成した「書籍情報の登録」リンクから遷移してきます)*
   - フォームの 送信先(action): `/book/register` 、 method: `POST`
   - 入力項目は、Formクラスのプロパティに合わせて `name` 属性を `title` 、 `author` 、 `price` に設定してください。
4. `BookController` に `/book/register` 向けの POST マッピングメソッドを作成してください。
5. 引数に `BookForm` を指定してデータを受け取り、受け取ったFormオブジェクトをそのまま `Model` に格納してください。
6. 遷移先画面として `book_confirm.html` （登録完了画面のHTMLファイル）を作成し、「以下の内容で登録を受け付けました（※実際のDB保存は次章で実装します）」というメッセージと共に、入力されたタイトル・著者名・価格を表示してください。

演習3：Spring Data JPA（データベースとの連携）

> **【学習のねらい】**
> 第3章の知識（Entity、Repository、Service、CRUD）と、一部の5・6章知識を使って、DBと連動する完全な書籍管理機能を作成します。

データベースのテーブル定義

本システムでは、以下の3つのテーブルを使用します。エンティティを定義する際の参考にしてください。

■ `genre` テーブル（ジャンル）

列名	データ型	制約	備考
id	INT	PRIMARY KEY, AUTO_INCREMENT	ジャンルID
name	VARCHAR(50)	NOT NULL	ジャンル名

■ `book` テーブル（書籍）

列名	データ型	制約	備考
id	INT	PRIMARY KEY, AUTO_INCREMENT	書籍ID
title	VARCHAR(255)	NOT NULL	書籍名
author	VARCHAR(100)	NOT NULL	著者名
price	INT	NOT NULL	価格
genre_id	INT	FOREIGN KEY → genre(id)	ジャンルID（外部参照）

■ `user` テーブル（ユーザ） ※第4章で使用

列名	データ型	制約	備考
user_id	INT(6)	PRIMARY KEY	ユーザID
password	VARCHAR(255)	NOT NULL	パスワード

列名	データ型	制約	備考
user_name	VARCHAR(50)	NOT NULL	ユーザ名

> **【注意】**
> `book` テーブルの `genre_id` は、`genre` テーブルの `id` を参照する**外部キー**です。エンティティ定義時にJPAのリレーションアノテーション（`@ManyToOne` / `@OneToMany`）を使って関連付ける必要があります（テキスト **第6章** 参照）。

課題3.1：エンティティクラスの作成

データベースのテーブルに対応するEntityクラスを作成しましょう。

1. `jp.co.trainocate.book.entity` パッケージを作成してください。
2. 上記の **`genre` テーブル** に対応する `Genre` エンティティクラスを作成してください。
   - `@Entity`、`@Data`（Lombok）、`@Table(name = "genre")` を付与します。
   - 各フィールドに `@Id`、`@GeneratedValue`、`@Column` 等の適切なアノテーションを付けてください。
   - **【6章の内容】** 1つのジャンルに対して複数の書籍が紐づきます。`@OneToMany(mappedBy = "genre")` を使い、`List<Book>` 型のフィールドを追加してください。
3. 上記の **`book` テーブル** に対応する `Book` エンティティクラスを作成してください。
   - `@Entity`、`@Data`（Lombok）、`@Table(name = "book")` を付与します。
   - 各フィールドに適切なアノテーションを付けてください。
   - **【6章の内容】** 複数の書籍は1つのジャンルに属します。以下の2つのフィールドを定義してください。
     - `@ManyToOne` と `@JoinColumn` を使って `Genre` 型のフィールド（ジャンルオブジェクト）を定義
     - `@Column(name = "genre_id")` を使って `Integer` 型のフィールド（ジャンルIDの値そのもの）を定義

> **【重要（テキスト6.3章参照）】**
> 上記のように「ジャンルオブジェクト」と「ジャンルID（数値）」の両方を同じテーブルの `genre_id` カラムにマッピングすると、登録や更新の際に「どちらの値を優先して保存すればいいのか」が分からずエラーになります。これを防ぐため、`@JoinColumn` 側に `insertable = false, updatable = false` を追記して、**「ジャンルオブジェクトの側は読み取り専用」**であることを明示してください。
> （例：`@JoinColumn(name = "genre_id", insertable = false, updatable = false)`）

課題3.2：リポジトリインターフェースの作成

エンティティに対するデータベース操作を行うRepositoryを定義しましょう。
これから実装する機能（全件取得、キーワード検索、価格帯検索、IDでの1件検索、登録・更新・削除）を踏まえ、リポジトリを作成します。

1. `jp.co.trainocate.book.repository` パッケージを作成してください。
2. `BookRepository` インターフェース（`JpaRepository<Book, Integer>`を継承）を作成してください。
3. 以下の検索に必要なメソッドを、**メソッド命名規則**（テキスト第3章）に従って定義してください。
   - タイトルに特定の文字列を含む書籍を検索するメソッド（`findByTitleContaining`）
   - 価格が指定範囲内の書籍を検索するメソッド（`findByPriceBetween`）
     (※全件取得(findAll)、IDでの1件取得(findById)、保存(save)、削除(deleteById)はJpaRepositoryに標準で存在するため自作不要です)
4. `GenreRepository` インターフェース（`JpaRepository<Genre, Integer>`を継承）を作成してください。

課題3.3：Service層の作成

RepositoryをControllerから直接呼ばず、Service層を介してアクセスするようにします。
本構成は、テキスト第3章の末尾にも登場する実践的な設計パターンです。

1. `jp.co.trainocate.book.service` パッケージを作成してください。
2. 以下のメソッドを定義したインターフェース `BookService` を作成してください。
   - `List<Book> findAllBooks()`
   - `Book findBookById(Integer id)`
   - `List<Book> findBooksByTitle(String title)`
   - `List<Book> findBooksByPrice(Integer minPrice, Integer maxPrice)`
   - `Book saveBook(BookForm bookForm)`
   - `void deleteBook(Integer id)`
3. `BookService` を実装した `BookServiceImpl` クラスを作成し、`@Service` アノテーションを付与してください。
   - **依存性の注入**: データベース操作を行うため、`BookRepository` をこのクラス内で利用できるようにします。フィールドとして宣言し、`@Autowired` を付与する（または Lombokの `@RequiredArgsConstructor` と `final` 制約を併用する）ことで、Springに「依存性の注入」を行わせてください。
   - **メソッドの実装（オーバーライド）**：インターフェースで定義した各メソッドの中身を書いていきます。このクラスはControllerとRepositoryの「橋渡し」となるため、取得した `BookRepository` が持つメソッド（`findAll()`, `findById()`, 課題3.2で定義した検索メソッド等）を呼び出し、その結果を return するように実装します。（登録処理 `saveBook` については、Formクラスの値をEntityにセットしてからRepositoryの `save()` を呼ぶ処理を記述してください）

課題3.4：全件一覧と検索モックのDB連動化

第2章で作ったモック画面を、Serviceを経由してDBから取得するように修正します。

1. `BookController` に `BookService` を「依存性の注入」してください。
2. `/book/list` メソッドを修正し、`BookService.findAllBooks()` を呼び出して全件リストを取得し、`Model` に渡します。
3. `/book/search/title` および `/book/search/price` メソッドを修正し、それぞれServiceから検索結果リストを取得して `Model` に渡します。
4. `book_list.html` と `book_search_result.html` を改修し、Thymeleaf の `th:each` を用いて、書籍を一覧表（ID、書籍名、著者名、価格、ジャンル名）で表示するようにしてください。
   - **【ヒント（外部参照の表示）】** `Book` エンティティが `Genre` エンティティを保持している場合、ジャンル名は `${book.genre.name}` のようにドット記法で階層を辿って表示できます。
   - **【重要（動的URL）】** 一覧の「書籍名」部分をリンクにし、クリックすると「詳細画面」へ飛ぶようにします。テキスト第5章のサンプルを参考に、以下のような**動的URL**にしてください。
     `<a th:href="@{/book/detail/} + ${book.id}" th:text="${book.title}"></a>`

課題3.5：書籍詳細画面の実装（更新・削除の起点）

書籍名をクリックしたときに表示される詳細画面を作成します。この画面から「更新」と「削除」を行えるようにします。

1. `BookController` に、動的URL `/book/detail/{id}` を受け取る GET マッピングメソッドを作成してください。（`@PathVariable` を使います）
2. Serviceの `findBookById(id)` を使って対象の書籍を1件取得し、 `Model` に格納してください。
3. 詳細画面 `book_detail.html` を作成し、ID、書籍名、著者名、価格、ジャンル名を表示してください。
4. この画面内に以下の2つのリンク（フォームのボタン）を配置してください。
   - **更新** （遷移先: `/book/update/{id}` へ遷移するフォーム `<form th:action="@{/book/update/} + ${book.id}">`）
   - **削除** （遷移先: `/book/delete/{id}` へ遷移するフォーム `<form th:action="@{/book/delete/} + ${book.id}">`）

課題3.6：新規登録機能の実装（モックからDB保存へ）

第2章で作成した機能を改修し、画面から入力された書籍データを実際にDBへ登録する一連のフローを完成させます。

1. **入力画面からの送信**
   - 登録処理は、メニュー画面（`book_index.html`）の「書籍情報の登録」リンクから、入力画面（`book_form.html`）へ遷移することで開始します。
   - `BookForm` クラスに `Integer genreId` フィールドを追加してください。
   - `book_form.html` のフォーム（action: `/book/register`, method: POST）に入力項目を追加し、送信データを受け取る準備をします。

> **【ヒント】** 外部参照しているジャンルデータについて、DBから一覧を取得してプルダウン（select要素）で選ばせる方法は、**第6章**で学習します。今回は簡易的に `genreId` を手入力する数値入力欄（`<input type="number" name="genreId">` など）として作成しておきましょう。

- 数値入力欄のそばには、以下のようなテキストを固定値で表示して、受講者が「どの数値を入力すればよいか」迷わないように工夫してください。
（参考）ジャンルID一覧： 1=プログラミング, 2=ビジネス, 3=小説, 4=科学, 5=自己啓発

2. **登録処理の実行**
   - `BookController` の `/book/register` (POST) メソッドの中身を修正します。
   - フォームから受け取った `BookForm` のデータを `Book` エンティティに移し替え、`BookService.saveBook()` を呼び出してDBに登録してください。
   - 登録処理が完了したら、結果を表示するために `book_confirm.html` へ画面遷移させます。その際、登録した書籍情報を `Model` に格納してください。
3. **登録完了画面の表示**
   - `book_confirm.html` を改修します。第2章で記述したモック用の説明文を削除し、「以下の内容で登録しました」というメッセージと共に、DBに登録された書籍情報（タイトル・著者名・価格・ジャンルID等）を表示してください。
   - 画面の下部に「メニューへ戻る」リンク（遷移先: `/book/index`）が配置されていることを確認してください。

課題3.7：更新機能と削除機能の実装

詳細画面から呼び出される更新と削除の処理を実装します。

1. **更新画面の表示**
   - `BookController` に `/book/update/{id}` を受け取る GET マッピングを追加してください。
   - DBから対象の `id` で書籍情報を取得し、その値を保持した更新用フォーム画面 `book_update.html` へ遷移させます。
   - `book_update.html` を作成し、該当の書籍情報が入力欄にセットされた状態のフォームを作成してください。
   - *(※課題3.6と同様に、ジャンルID（`genreId`）の入力欄と、入力を補助する「ジャンルID一覧」のテキスト表示も忘れずに配置してください)*
2. **更新の実行（動的URLの利用）**
   - `BookForm` に `id` フィールドを追加してください。
   - 更新用フォームの送信先（action）について、今回は `@PathVariable` を用いてIDをURLに含めて送信します。テキスト第5章にならい、`<form th:action="@{/book/update/} + ${bookForm.id}" method="post">` のように記述してください。
   - `BookController` に `/book/update/{id}` を受け取る POST マッピングを追加します。引数で `@PathVariable` を使ってIDを受け取り、フォームからの値と共にエンティティにセットしてから `BookService.saveBook()` を呼び出します（IDが存在するためUPDATEとして働きます）。
   - 処理完了後は、詳細画面や結果画面へ遷移させてください。
3. **削除の実行**
   - `BookController` に `/book/delete/{id}` を受け取るマッピングを追加し、Serviceの `deleteBook(id)` を呼び出します。
   - 削除完了後は、全件リスト（`/book/list`）へ **リダイレクト** するようにしてください。

第4章：入力チェックとログイン認証（セッション管理）

> **【学習のねらい】**
> 第3章で作成したCRUDアプリに対して、**値の入力チェック（バリデーション）**と、**DBを利用したユーザー認証およびセッション** の仕組みを導入します。

課題4.1：入力チェック（Validation）の実装

ユーザーが不正なデータを入力した際に、エラーメッセージと共に元の画面へ戻す処理を実装します。

> **【重要】バリデーション対象について**
> 本来、セキュリティの観点からはログイン画面（`index.html`）に対しても入力チェック（例：ユーザーIDが空、パスワードが短すぎる等）を実装すべきですが、本演習では実習時間の関係上、**書籍の登録・更新画面にのみバリデーションを実装します。** ログイン画面については、DB認証のみで判定を行います。

1. **フォームクラス (`BookForm`) へのアノテーション付与**
   - 以下のフィールドに対して入力チェックのアノテーション（`javax.validation` あるいは `jakarta.validation`）と、エラーメッセージ（`message` 属性）を付与してください。
     - `title`：空白不可（`@NotBlank`）。メッセージ：「書籍名を入力してください」
     - `author`：空白不可（`@NotBlank`）。メッセージ：「著者名を入力してください」
     - `price`：空白不可（`@NotNull` / メッセージ：「価格を入力してください」）、かつ 0 以上（`@Min(0)` / メッセージ：「価格は0以上で入力してください」）
     - `genreId`：空白不可（`@NotNull`）。メッセージ：「ジャンルIDを入力してください」
2. **Controllerのエラー処理実装**
   - `BookController` の `/book/register` (POST) と `/book/update/{id}` (POST) メソッドを修正します。
   - 引数の `BookForm` に `@Valid` を付与し、その直後に `BindingResult result` を受け取るようにします。
   - `result.hasErrors()` が `true` の場合、元の入力画面（テンプレート）の名前を `return` する処理を記述してください。
3. **HTMLでのエラー表示**
   - `book_form.html` と `book_update.html` を修正します。
   - `<form>` 要素に `th:object="${bookForm}"` を設定し、各入力欄（`title` など）の付近に `<div th:errors="*{title}" class="text-danger"></div>` というエラー表示用の要素を追加して

ください。

課題4.2：ユーザー管理用のEntityとRepository作成

データベースに存在する `user` テーブルを利用してログイン認証を行うための準備をします。

1. **`User` エンティティの作成**
   - `jp.co.trainocate.book.entity.User` クラスを作成してください。
   - DBの `user` テーブル（カラム: `user_id`, `password`, `user_name`）と一致するようにフィールドを定義し、アノテーション（`@Entity`, `@Table`, `@Id` など）を付与します。
2. **`UserRepository` の作成**
   - `JpaRepository` を継承したインターフェースを作成します（主キーは Integer に合わせてください）。
   - ログイン認証で利用するため、ユーザIDとパスワードの両方が一致するデータを検索するメソッド `User findByUserIdAndPassword(Integer userId, String password);` を追加してください。

課題4.3：ログイン・ログアウト機能の追加

セッションを用いたログイン認証と、ログアウト機能を実装します。第4章ではまだ条件分岐（`th:if`）を使いませんので、ログイン状態に関わらず要素を並べて配置すること（例：`<span>` や `<a>` を並べる）を優先してください。

1. **HTMLでのセッション情報表示とログアウトリンク**
   - `book_index.html` や `book_list.html` などの主要な画面上部に、「ようこそ、〇〇さん」と表示する領域を追加してください。
   - そのすぐ横に、ログアウト画面（`/logout`）へのリンクを配置してください。
   - **【ヒント1】** Thymeleaf では、セッションの属性に `${session.userName}` のようにアクセスできます。
   - **【ヒント2】** ログインしていない状態でも「ようこそ、 さん」と表示されたり、ログアウトリンクが見えたりしても問題ありません。
2. **LoginController の改修（認証とログアウトの実装）**
   - `/login` に対するPOSTメソッドを変更し、`UserRepository` を使ってDB認証を行います。ログイン成功時は `session.setAttribute("userName", user.getUserName())` でセッションに値を保持してください。
   - **認証失敗時**は、`model.addAttribute("error", "ユーザーIDまたはパスワードが違います")` を実行してエラーメッセージをセットし、ログイン画面（`return "index";`）を再表示するようにしてください。
   - 新たに、ログアウト用（`/logout`）のGETマッピングメソッドを作成してください。
   - 引数で `HttpSession` を受け取り、`session.invalidate()` を実行してセッションを破棄した後、ログイン画面（`redirect:/`）へリダイレクトさせてください。
3. **ログイン画面の改修**
   - `index.html` 内に、認証エラー時に `model` から渡された `error` メッセージを赤字などで表示するようにしてください。

演習5：共通レイアウトの適用とデザインの統一（Thymeleaf Layout Dialect）

> **【学習のねらい】**
> これまで各HTMLファイルにバラバラに記述していた共通部分（ヘッダー、フッター、ログイン情報の表示など）を、「共通レイアウト」として1箇所にまとめます。これにより、デザイン変更が容易になり、各画面のコードもスッキリと管理できるようになります（**DRY原則**の実践）。
> 本章では、外部ライブラリの **Thymeleaf Layout Dialect** を使用して、テンプレートの継承構造を構築します。

課題5.1：ライブラリの導入

レイアウト機能を利用するために、`pom.xml` に依存関係を追加しましょう。

1. `pom.xml` の `<dependencies>` セクション内に、以下の依存関係を追加してください。追加後は Maven プロジェクトの更新（Reload）を忘れずに行ってください。

```
<dependency>
    <groupId>nz.net.ultraq.thymeleaf</groupId>
    <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

共通レイアウトの配置確認

開発の効率化とデザイン統一のため、配布されたレイアウト部品がプロジェクト内に正しく配置されているか確認します。

1. **CSSの確認**: `src/main/resources/static/css/` に `style.css` が配置されていることを確認してください。
2. **レイアウトの確認**: `src/main/resources/templates/layout/` に `layout.html` が配置されていることを確認してください。
   - **【解説】** `layout.html` はシステム全体の「枠組み」です。ヘッダーには「ようこそ、〇〇さん」といったセッション情報の表示も含まれています。
   - **【確認】** `layout.html` 内に、各画面の中身が挿入されるための領域 `<main layout:fragment="content">` が定義されていることを確認してください。

課題5.2：全画面へのレイアウト適用（リファクタリング）

作成済みの全HTML画面（8画面分）を、共通レイアウトを「継承」する形に書き換えます。

1. **テンプレートの宣言修正**
   - 各HTMLの `<html>` タグを、以下のように修正してください。

     ```
     <html xmlns:th="http://www.thymeleaf.org"
           xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
           layout:decorate="~{layout/layout}">
     ```

   - `layout:decorate` は、「このファイルは `layout/layout.html` を親（型紙）として使います」という宣言です。
2. **コンテンツの埋め込み**
   - 各画面の `<body>` 内にあった具体的なコンテンツ（`<h1>` や `<form>`、`<table>` など）を、一つのタグで囲い、以下のアノテーションを付与してください。

     ```
     <main layout:fragment="content">
         <!-- ここに元のコンテンツを入れる -->
     </main>
     ```

   - これにより、このタグの中身だけが `layout.html` の `<main>` 部分に自動的に流し込まれます。
3. **重複コードの削除**
   - 共通レイアウト側で定義されている内容は、各ファイルからは **削除** してください。
   - メタタグやCSSの読み込み記述、ヘッダーの情報（「ようこそ、〇〇さん」やログアウトボタン等）はレイアウト側に集約されるため、各画面のコードは非常にスッキリします。

動作確認

1. ログイン画面（`http://localhost:8080/`）からログインし、全画面でレイアウトが統一されていることを確認してください。
2. ヘッダーにログインユーザ名が正しく表示され続け、機能が正しく動作することを確認してください。

課題5.3：ログイン・ログアウトボタンの切り替え（th:ifの活用）

これまでは固定で「ログアウト」ボタンが表示されていましたが、ログイン状態に応じて適切に表示を切り替えるようにします。

1. `layout.html` 内のヘッダー部分を修正します。
2. `th:if="${session.userName == null}"` を使用して、未ログイン時は「ログイン」リンクを表示します。
3. `th:if="${session.userName != null}"` を使用して、ログイン済みなら「ようこそ表示 ＋ ログアウトボタン」を表示するように制御してください。

演習6：高度なデータ連携とユーザー体験の向上（動的な項目取得と0件制御）

> **【学習のねらい】**
> Webアプリケーションとしての完成度を高めます。検索結果がなかった場合の適切なメッセージ表示（0件制御）や、DBから取得した動的なリスト（ジャンル一覧）をフォームの選択肢として利用する方法を学びます。
> また、より複雑な検索条件に対応するため、Repository層で **JPQL** を活用する手法も習得します。

課題6.1：検索結果0件時の表示制御

書籍が見つからない場合に、真っ白な画面ではなく「該当する書籍はありません」という案内を出すように改修します。

1. `book_list.html` および `book_search_result.html` を修正してください。
2. リスト（`books`）が空の場合にのみ表示されるメッセージ領域を追加します。
    - **【ヒント】** Thymeleaf の `th:if="${#lists.isEmpty(books)}"` を使用すると、リストが空の時だけ要素を表示できます。
    - **【表示例】** 第5章で用意された CSS のスタイルなどを利用して、ユーザーに分かりやすく表示しましょう。

課題6.2：登録画面での動的なジャンル選択（GenreServiceの作成）

これまではジャンルIDを数字で手入力していましたが、DBから取得したジャンル名の一覧をセレクトボックス（プルダウン）で選べるようにします。

1. **GenreService の作成**
   - `Genre` エンティティを操作するための `GenreService` インターフェースと、その実装クラス `GenreServiceImpl` を作成してください。
   - DBにあるすべてのジャンルをリストで取得する `findAllGenres()` メソッドを実装します。
2. **Controller でのデータ準備**
   - `BookController` に `GenreService` を注入してください。
   - 登録画面を表示する GET メソッド（`/book/form`）において、Service から全ジャンルのリストを取得し、`Model` に格納してください（属性名は `genres` など）。
3. **HTML の改修**
   - `book_form.html` の `genreId` 入力欄（input）を、セレクトボックス（`<select>`）に変更します。
   - `th:each` を使って、取得したジャンルリストから `<option>` タグを動的に生成してください。

```
<select th:field="*{genreId}">
    <option th:each="g : ${genres}" th:value="${g.id}" th:text="${g.name}"></option>
</select>
```

課題6.3：【発展】JPQLを用いた複合検索の実装

単純な「名前だけ」の検索ではなく、複数の条件を組み合わせた複雑な検索を、Repository層に自分でクエリを書いて実装します。

1. **Repository へのメソッド追加**
   - `BookRepository` に、JPQLを使ったカスタム検索メソッドを作成してください。
   - **【課題内容】** ジャンル名（Name）の一部一致（LIKE） **かつ** 指定された価格（Price）以下 の書籍を検索する機能を実装します。
   - **【クエリ例】**
     `SELECT b FROM Book b JOIN b.genre g WHERE g.name LIKE %:genreName% AND b.price <= :maxPrice`

> **【なぜJPQLを使うのか？】**
> 今回の課題のように「外部参照先のテーブルの項目（Genre名の部分一致）」と「自テーブルの項目（価格の範囲指定）」を組み合わせる場合、Spring Data JPAのメソッド名自動生成機能では `findByGenreNameContainingAndPriceLessThanEqual` のようになり、名前が非常に長く、管理が難しくなります。JPQLを使用することで、複雑な条件を簡潔かつ読みやすく記述することができます。

2. **検索機能の追加**
   - Service と Controller にこの検索機能を組み込んでください。
   - **【確認】** ジャンル名（例：プロ）と上限価格（例：3000）を入力した際に、条件を満たす書籍のみが抽出されることを確認しましょう。

課題6.4：【オプション】更新画面への動的リスト適用

課題6.2と同様の改修を、書籍情報の更新画面（`book_update.html`）にも適用してください。

- 画面遷移時に正しく現在のジャンルが初期選択（selected）されることを確認しましょう。

第7章： デザインの洗練（Bootstrapの導入）

【学習のねらい】
これまでは個々のCSSプロパティを使ってデザインを行ってきましたが、実務では **Bootstrap** のようなCSSフレームワークを利用して、迅速かつ洗練されたデザインを適用するのが一般的です。
本章では、既存の「書籍管理システム」に Bootstrap を導入し、プロフェッショナルな外観のWebアプリケーションへとアップグレードします。
学習のステップとして、まずはボタンやフォームといった小さな部品から改善し、最後に全体のレイアウト（グリッドシステム）を整えていきます。

第7章でよく使う Bootstrap クラス早見表

これから各課題で使用する主なクラスの役割です。この表を参考に、デザインの変化を「体験」してみましょう。

カテゴリ	クラス名	役割 (説明)
レイアウト	`container`	コンテンツを中央に寄せ、左右に適切な余白を作ります。
	`row`	グリッド（12分割）を作るための「行」です。
	`col-md-X`	画面幅に応じた「列」を作ります（Xは1〜12の数値）。
ボタン	`btn`	要素をボタンらしい見た目にします。
	`btn-primary`	主要な操作（青色）を設定します。
	`btn-secondary`	戻るなどの控えめな操作（灰色）を設定します。
	`btn-danger`	削除などの危険な操作（赤色）を設定します。
フォーム	`form-label`	入力項目のラベル用のスタイルを適用します。
	`form-control`	テキストボックス等の入力欄をモダンな見た目にします。
	`form-select`	セレクトボックス（プルダウン）用のスタイルです。
テーブル	`table`	テーブルの基本、および縞模様（`table-striped`） 等のスタイルです。

カテゴリ	クラス名	役割 (説明)
共通設定	`mb-3`	下方向に適度な余白（Margin Bottom）を作ります。
	`p-4`	内側に余白（Padding）を作ります。
	`bg-white`	背景色を白（Background White）にします。
	`rounded`	角を丸く（Rounded）します。
	`shadow-sm`	小さな影（Shadow Small）を付け、コンテンツを際立たせます。

【ヒント： d-flex クラスについて】

`d-flex` クラスを使うと、要素を簡単に横並びに配置できます（CSSの Flexbox という機能を利用しています）。

- `d-flex` ： その要素の中身を「横並び」に変えます。
- `align-items-center` ： 横並びにした要素たちの「上下中央」を揃えます。
- `gap-2` ： 要素と要素の間に「適度なスキマ」を作ります。

レイアウトの調整に迷ったら、この3点セットを思い出してみてください！

課題7.1： Bootstrap の導入と既存CSSの排除

Spring Boot プロジェクトに Bootstrap を導入し、デザイン刷新の準備をします。

【対象ファイル】： `templates/layout/layout.html`

1. 共通レイアウト（`layout.html`）の `<head>` タグ内に、Bootstrap の CSS を読み込む記述を追加してください。
2. **【重要】** これまでデザインを担当していた `style.css` の読み込み（`<link rel="stylesheet" th:href="@{/css/style.css}">`）を 削除 してください。
   - **【解説】** これにより、独自CSSの設定（第5章で作成したもの）との干渉がなくなり、Bootstrap 本来の挙動とデザインを純粋に体験できるようになります。
3. `<body>` タグの末尾（`</body>` の直前）に、Bootstrap の JavaScript を読み込む記述を追加してください。

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></s
```

課題7.2： ボタンのデザイン統一

画面内のリンクや送信ボタンに Bootstrap のボタンクラスを適用します。

【対象ファイル】： 全ての HTML ファイル (`index.html`, `book_index.html`, `book_list.html`, `book_detail.html`, `book_confirm.html`, `book_form.html`, `book_update.html`, `book_search_result.html`)

1. システム内の各画面にある全ての `<a>` タグや `<button>` タグに `btn` クラスを付与してください。
2. 動作や役割に応じて `btn-primary` (登録・ログイン)、`btn-secondary` (戻る)、`btn-danger` (削除) などを使い分けてください。
   - 詳細は冒頭の「早見表」を参照しましょう。

課題7.3： フォーム部品の整形

入力項目を Bootstrap のスタイルに変更し、余白（`mb-3`）やラベル（`form-label`）、入力欄（`form-control`）を整えます。

【対象ファイル】： `index.html`, `book_index.html`, `book_form.html`, `book_update.html`, `book_search_result.html`

1. 各入力画面のフォーム部品を、Bootstrap のルール（`mb-3`, `form-label`, `form-control`, `form-select`）に従って書き換えてください。
2. エラーメッセージの表示箇所には `text-danger` クラスを適用し、赤字で表示されるようにします。

課題7.4： テーブル（書籍一覧）の整形

書籍リストや検索結果のテーブルに `table`, `table-striped`, `table-hover` を適用し、 `<thead>` には `table-dark` を指定してください。

【対象ファイル】： `book_list.html`, `book_search_result.html`, `book_detail.html`, `book_confirm.html`

1. 各画面の `<table>` タグに、Bootstrap のテーブル関連クラスを付与してください。
2. `<thead>` タグを `table-dark` クラスで囲み、ヘッダー部分を強調しましょう。

課題7.5： 全体のレイアウト刷新

最後に、Bootstrap の Navbar やコンテナを使用して、システム全体の枠組みをプロフェッショナルな外観に整えます。

【対象ファイル】： `templates/layout/layout.html`, `index.html`, `book_index.html`, `book_form.html`, `book_update.html`

1. **共通レイアウト（layout.html）の修正**
   ヘッダー、ナビゲーション、フッターを Bootstrap のクラスで書き換え、メインコンテンツを `container` で包みます。

```html
<body class="bg-light">
    <header>
        <nav class="navbar navbar-expand-lg navbar-dark bg-dark shadow-sm">
            <div class="container">
                <a class="navbar-brand fw-bold" th:href="@{/book/index}">TrainoBook</a>
                <!-- ここに「ようこそ、〇〇さん」やログアウトボタンを配置 -->
            </div>
        </nav>
    </header>
    <div class="container my-5">
        <!-- mainタグ自体に背景色や余白のクラスを付与 -->
        <main layout:fragment="content" class="p-4 bg-white rounded shadow-sm">
            <!-- 各画面の中身がここに挿入される -->
        </main>
    </div>
    <footer class="py-4 bg-white border-top text-center text-secondary">
        <p class="mb-0">&copy; 2026 Trainocate Book Management System</p>
    </footer>
</body>
```

2. **グリッドシステム（配置制御）による画面の整形**
   `row` と `col` クラスを使い、ログインフォームを中央に寄せたり、検索フォームを横並びに配置したりして、バランスを整えましょう（ログイン画面、登録・更新画面、メニュー画面等）。
3. **【最終確認】**
   全ての画面が Bootstrap によって美しく整えられ、ブラウザの幅を変えても崩れないレスポンシブなデザインになっていることを確認して、全演習完了です！お疲れ様でした！